Bangladesh Transport Infrastructure Case

Assignment 3: Network Model Generation

EPA133A: Advanced Simulation

Team 14

Delft University of Technology

TU Delft

Bangladesh Transport Infrastructure Case

Assignment 3: Network Model Generation

by

Team 14

Bayu Jamalullael	6367984
Brenda Escobar	6512191
Brian Parsaoran	6147674
Taufik M. Yusup	6378056
Zhafran Sidik	5658411

20 March 2026

Faculty of Technology, Policy and Management (TPM)

TU Delft

Contents

1 Introduction **1**
1.1 Context and Motivation . . . 1
1.2 Goal and Scope of This Assignment . . . 1

2 Data Preparation and Network Generation **2**
2.1 Dataset Description . . . 2
2.2 Creating the Model-Ready CSV . . . 2
2.3 Network Summary and Visualisation . . . 3
2.4 Processing Functions . . . 5
2.4.1 N-Road Name Extraction - `extract_nroads()` . . . 5
2.4.2 Side-Road Discovery - `identify_intersecting_sideroads()` . . . 6
2.4.3 Distance Utility - `haversine_distance()` . . . 6
2.4.4 Spatial Matching - `find_nearest_bridge_on_road()` . . . 6
2.4.5 Network Construction - `build_simulation_network()` . . . 6
2.4.6 Intersection Unification - `unify_intersection_identifiers()` . . . 6
2.4.7 Distance Computation - `compute_inter_node_distances()` . . . 7

3 Model Design and Implementation **8**
3.1 Updates in `model.py` . . . 8
3.1.1 Reading the roads csv and processing into a network graph . . . 8
3.1.2 Picking routes and computing shortest path . . . 9
3.1.3 Recording trip data analytics . . . 9
3.2 Updates in `components.py` . . . 9
3.3 Updates in `model_run.py` . . . 9
3.4 Reflection on the model design . . . 9

4 Results and Discussion **12**
4.1 Experimental Setup . . . 12
4.2 Results . . . 12
4.3 Bonus Questions . . . 15
4.3.1 Bonus 1: Accurate Intersection Locations Using Road Geometries . . . 15
4.3.2 Bonus 2: Impact of Intersection Location Accuracy on Experimental Outcomes . 18

5 Conclusion and Reflection **20**
5.1 Conclusion . . . 20
5.2 Limitations . . . 20
5.3 Future Improvements . . . 21

References **22**

A Data and Code Availability **23**

B Use of AI Tools **24**
B.1 How AI Was Used . . . 24
B.2 How AI Was Not Used . . . 24

C Team Contributions **25**

1 Introduction

1.1. Context and Motivation

The N1 and N2 highways, along with their connected national roads are fundamental components of Bangladesh's primary highway network and play an crucial role in economic activity. The N1 links the capital Dhaka with Chittagong, the country's principal seaport, and facilitates movement of millions of containers each year. The N2 connects Dhaka to Sylhet and extends to Tamabil at the Indian border, supporting regional connectivity and cross-border movement. Together, these routes facilitate the movement of goods and people.

In the previous report, only the N1 was modeled to evaluate the most critical bridges along the road under specific simulation conditions. The component-based approach used allows expansion of the model toward a network model that incorporates the N1, N2, and their connected national roads.

Roads are often modeled as networks because they usually consist of a system of junctions (nodes) and segments (links/arcs) that directly map to graph theory principles. Therefore, their structure and performance can be analyzed more efficiently.

1.2. Goal and Scope of This Assignment

The goal of this study is to automatically generate an agent-based model using Mesa, simulate the effects of bridge maintenance or unavailability on traffic throughout a road network that includes N1 and N2 roads and their side roads with more than 25 kilometers, and, based on the simulation results, select top five candidate bridges to invest.

The model simulates vehicles driving or waiting on the road. Each simulation run, random bridges can break down based on their condition and increase delay time. The network modeling allows two-way traffic along the roads and vehicles can travel through multiple roads. The model generated provides information on how bridge conditions can propagate delays along different routes. This document reports how the modeling process was carried out, the simulation experiments and the results obtained.

2

Data Preparation and Network Generation

This chapter describes the data sources and the processing pipeline that transforms raw survey records into `network_data.csv`, the direct input to the Mesa/NetworkX simulation model. Two datasets are combined through a six-step automated pipeline, producing a model-ready network spanning nine roads across Bangladesh's N1 and N2 main corridors.

2.1. Dataset Description

Two source files drive the entire pipeline.

`_roads3.csv` is a nation-wide road-survey file containing 51 348 Location Reference Point (LRP) records, each carrying eight attributes: **road** (identifier), **chainage** (cumulative km), **lrp** (unique code), **lat/lon** (WGS 84 coordinates), **gap** (`BS`/`BE` markers for bridge boundaries), **type** (e.g. *KmPost*, *Bridge*, *Culvert*, *SideRoad*, *CrossRoad*), and **name** (human-readable label).

`BMMS_overview.xlsx`, exported from Bangladesh's Bridge Management and Maintenance System (BMMS), records 20 415 structures with attributes including physical length, condition rating (A–D), coordinates, structure number, zone/division, and construction year. Jafino et al. [1] used this same BMMS database as the primary data source for their transport network criticality study in Bangladesh, confirming that the dataset is an established reference for bridge-level infrastructure analysis in the country. As with Assignment 2, many records were entered manually by field operators and may contain positional inaccuracies, dual-carriageway duplicates, or missing fields, data quality issues that Jafino et al. [1] also acknowledged when pre-processing the same dataset for their criticality computations.

For this assignment the model covers a substantially wider network than in A2: the two main arteries **N1** (Dhaka–Teknaf) and **N2** (Dhaka–Sylhet) are included together with every N-type side road that intersects either main road and whose total chainage exceeds 25 km. This corridor-based selection mirrors the Bangladesh freight transport network used by Jafino et al. [1] as their primary case study for comparing criticality metrics, making their findings a directly relevant benchmark for interpreting our simulation results. The resulting road list comprises nine roads: N1, N2, N102, N104, N105, N106, N204, N207, and N208.

2.2. Creating the Model-Ready CSV

A six-step pipeline implemented in `network_data_preparation.ipynb` transforms the two source files into `network_data.csv`:

1. **Side-road discovery** (`identify_intersecting_sideroads`). Scans `_roads3.csv` for `SideRoad`, `CrossRoad`, and `Others` type entries and extracts any N-road names mentioned in either the `road` or `name` column when the complementary column references N1 or N2. This yields a candidate

list of intersecting side roads.

2. **Length filtering** (`filter_roads_by_minimum_length`). Retains only those side roads whose maximum chainage in `_roads3.csv` exceeds 25 km, as required by the assignment. Roads below this threshold are discarded with a diagnostic message. N106 was not found automatically and was added manually via `add_missing_sideroads`.
3. **Network construction** (`build_simulation_network`). Iterates over every road in the combined list and classifies each LRP record as one of four model types: *sourcesink*, *link*, *bridge*, or *intersection*. Bridge records from `BMMS_overview.xlsx` are spatially matched using `find_nearest_bridge_on_road` and assigned condition ratings (A–D); Jafino et al. [1] use these same four condition categories to weight bridge criticality in their network-disruption scenarios, establishing condition A (good) through condition D (very poor) as the standard classification for Bangladesh's national road bridges. Unmatched structures default to condition `B`. Each component receives a unique integer ID starting at $r \times 10^6$, where r is the 1-based index of the road in the list.
4. **Intersection edge-case handling** (`handle_intersection_edge_cases`). Four categories of manually identified edge cases are resolved:
   (a) Sourcesink nodes that also serve as road intersections are given an additional *intersection* row so that the simulation model can recognise the crossing.
   (b) The N1–N106 junction lies inside a Chittagong city survey gap and is placed at the N106 LRPS coordinates ("Oxygen More", 22.3935552°N, 91.8215827°E). A dedicated intersection row is added to N106 alongside the existing LRPS sourcesink node, following the same pattern as N102 and N105; the N106 LRPS sourcesink retains its own auto-assigned ID (9 000 000). The corresponding N1 intersection node is inserted after LRP235b.
   (c) The N104–N1 junction: N104's LRPS entry references `Z1031` rather than `N1` in the survey data. A mirror intersection row is added to N104 at the same coordinates as the N1 crossing so that the unification step can pair them correctly.
   (d) The missing N105 LRPE sourcesink: the last N105 survey row is a `KmPost/Others` entry rather than a proper `LRPE` record, so the closing source/sink node is injected manually.
5. **Intersection ID unification** (`unify_intersection_identifiers`, `align_n1_n2_intersection_ids`). Paired intersection rows, one on each road at the same physical crossing, are detected by checking whether one row's `road` appears in the other's `name`, subject to a 500 m proximity threshold that absorbs GPS and chainage rounding errors. Matched pairs are assigned the lower of their two IDs. A dedicated pass then ensures the N1–N2 primary junction pair shares the same ID.
6. **Distance computation and export** (`compute_inter_node_distances`). Missing `length` values are filled with the Haversine distance from the preceding node on the same road; BMMS-supplied bridge spans and zero-length source/sink entries are preserved. All lengths are rounded to integer metres and the result is saved as `data/network_data.csv`.

The output contains one row per network component across all nine roads, with columns `road`, `id`, `model_type`, `name`, `lat`, `lon`, `length`, `condition`, and `lrp`.

2.3. Network Summary and Visualisation

Table 2.1 summarises the composition of the generated `network_data.csv` across all nine roads.

Figure 2.1 shows all nodes colour-coded by model type. Links form the road backbone; bridges (red triangles) are densely distributed along N1 and N2; source/sink nodes (green stars) mark road endpoints; and intersection diamonds (orange) mark the 13 physical junctions.

Table 2.1: Component counts in `network_data.csv`.

Model type	Rows	Notes
Link	3 302	Road segments between structural nodes.
Bridge	737	Matched to BMMS records; unmatched structures default to condition B.
SourceSink	18	Two per road (LRPS + LRPE) across nine roads.
Intersection	26	26 rows representing 13 unique physical junctions; each junction appears once per road it connects, with both rows sharing the lower of their two IDs after unification.
Total	**4 083**	

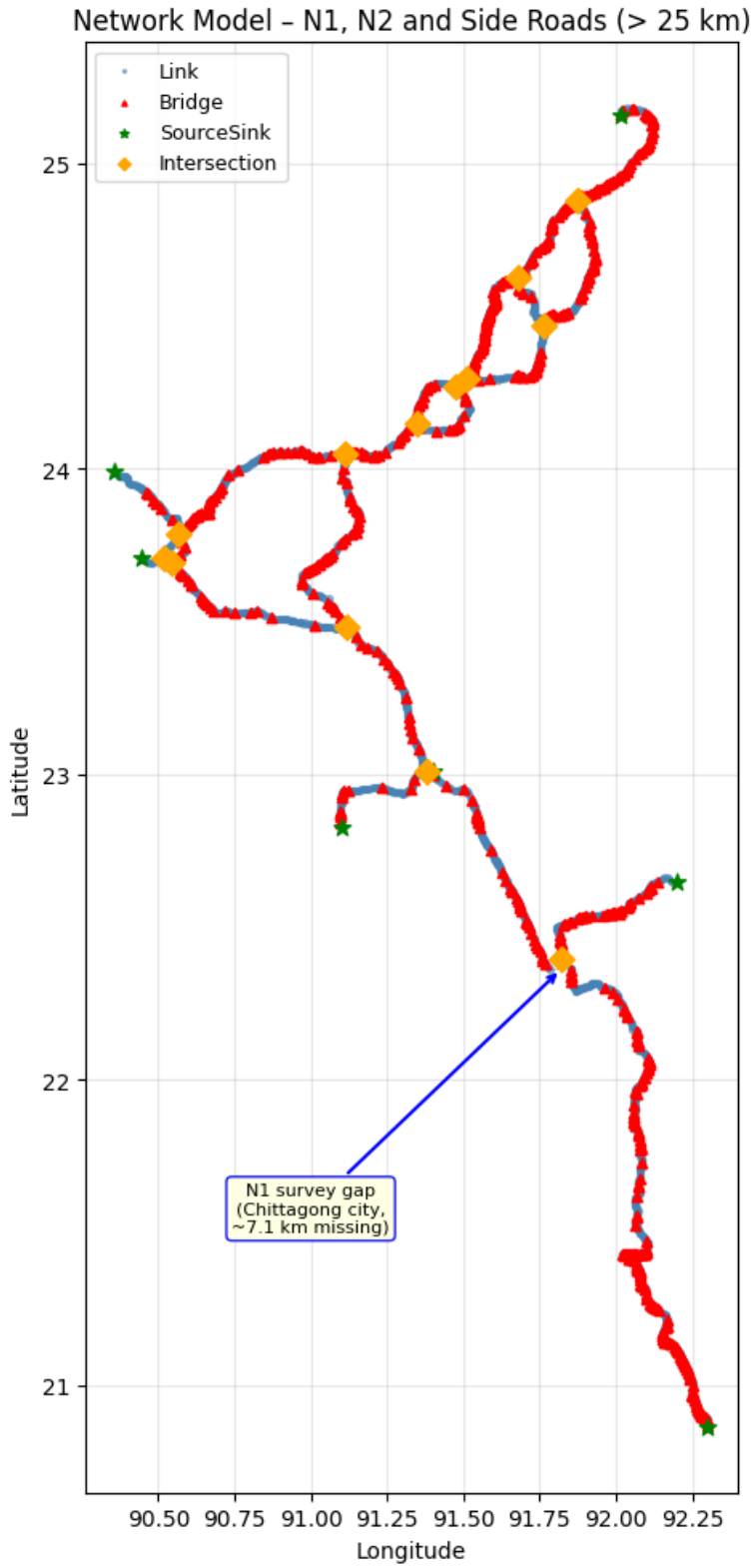

Figure 2.1: Network model node map for N1, N2, and the seven side roads (>25 km). Blue dots: links; red triangles: bridges; green stars: source/sink nodes; orange diamonds: intersections.

Figure 2.2 presents the same network with each road drawn in a distinct colour, road labels placed at midpoints, and only intersection and source/sink nodes annotated.

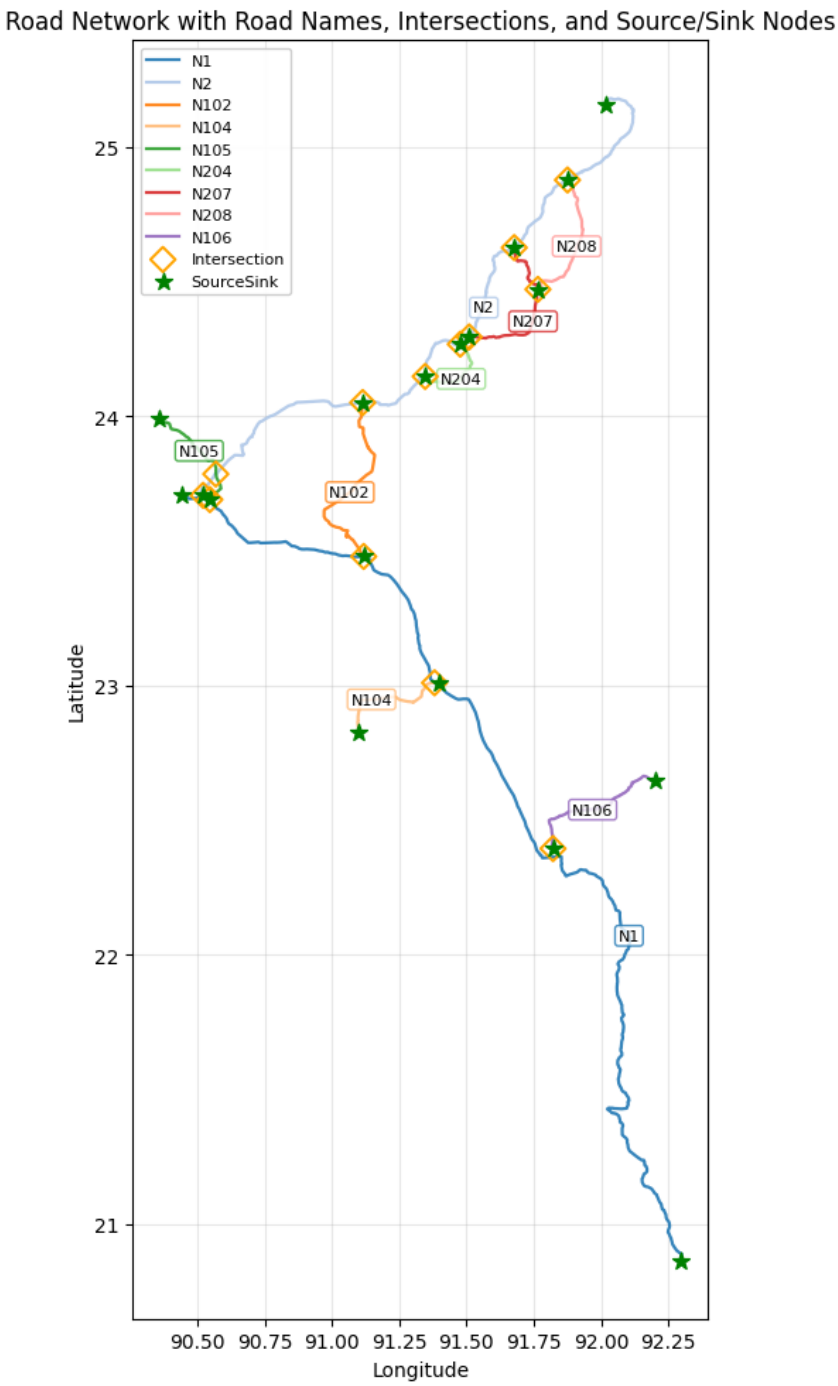

Figure 2.2: Road network with per-road colouring, road name labels, intersection nodes (orange diamonds), and source/sink nodes (green stars).

2.4. Processing Functions

2.4.1. N-Road Name Extraction - `extract_nroads()`

A regular expression (`\bN\s?\d{1,3}\b`) scans any text string and returns the set of N-road names it contains, normalising variants such as `"N 2"` or `"N 208"` to the canonical forms `"N2"` and `"N208"`.

This function is called by `identify_intersecting_sideroads` and by the intersection-unification logic to determine which roads a given LRP record references.

2.4.2. Side-Road Discovery - `identify_intersecting_sideroads()`

Iterates over all rows whose `type` contains `SideRoad`, `CrossRoad`, or `Others`. For each such row, `extract_nroads` is applied to both the `road` and `name` columns. If one column references a main road (N1 or N2) the function collects all N-road names from the other column as candidate side roads. The result is a de-duplicated, sorted list that seeds the road selection for the full network model.

2.4.3. Distance Utility - `haversine_distance()`

Computes the great-circle distance in metres between two WGS 84 points using the Haversine formula:

$$a = \sin^2\!\left(\frac{\Delta\varphi}{2}\right) + \cos\varphi_1 \, \cos\varphi_2 \, \sin^2\!\left(\frac{\Delta\lambda}{2}\right), \tag{2.1}$$

$$d = 2R \, \operatorname{atan2}\!\left(\sqrt{a}, \, \sqrt{1-a}\right), \tag{2.2}$$

where $R = 6{,}371{,}000$ m, φ is latitude, and λ is longitude. The function is called by the spatial-matching step, the distance-computation step, and the intersection-unification proximity check.

2.4.4. Spatial Matching - `find_nearest_bridge_on_road()`

Given a coordinate and a road name, performs a linear scan over the BMMS DataFrame, computes the Haversine distance from each bridge record to the query point, and returns the closest record together with its distance. Proximity thresholds applied in `build_simulation_network` are 50 m for culverts and 200 m for bridges. The tighter culvert threshold avoids false matches between the densely-spaced small structures (2–15 m span), while the wider bridge threshold accommodates the offset between the midpoint of a long bridge span and the abutment or centroid coordinate stored in BMMS.

2.4.5. Network Construction - `build_simulation_network()`

The central function that iterates over all roads and classifies each LRP record:

- **SourceSink.** `LRPS` (start) and `LRPE` (end) records become source/sink nodes with zero or NaN length. These serve as both vehicle generators and terminators in the Mesa simulation.
- **Culvert.** A link node is placed at the survey coordinate, followed by a bridge node if a BMMS record is found within 50 m. Culverts at the same chainage are grouped and noted in the bridge name.
- **Bridge (BS/BE pairs).** Start (`BS`) and end (`BE`) markers are consumed together. The bridge node is placed at the midpoint of the two markers; a BMMS match within 200 m supplies the real span length and condition rating. Unmatched bridges receive condition `B` and a span estimated from the BS–BE Haversine distance. A zero-length link node follows to avoid double-counting the span.
- **Intersection.** `CrossRoad` and `SideRoad` entries whose name references another road in the model are classified as intersection nodes.
- **Default (link).** All remaining record types (KmPost, monument, etc.) become plain link nodes connecting road segments.

Each component receives a unique ID from a per-road counter, starting at $r \times 10^6$ (where r is the 1-based road index), ensuring IDs never collide across roads.

2.4.6. Intersection Unification - `unify_intersection_identifiers()`

After the network is built, every road holds its own intersection row for each junction it participates in. This function pairs each such row with its counterpart on the crossing road and assigns both the lower of their two IDs, so that the simulation model can recognise the pair as a single shared node.

Two rows are paired when (a) one row's `road` value appears in the other's `name` field, and (b) they lie within 500 m of each other. The 500 m threshold is chosen to absorb typical GPS and chainage rounding errors in `_roads3.csv` ($\sim$100–300 m) while being small enough never to merge two distinct intersections, which are always several kilometres apart on Bangladesh's national road network.

2.4.7. Distance Computation - `compute_inter_node_distances()`

Fills every `NaN` length value in the final DataFrame with the Haversine distance from the preceding node on the same road. The first entry of each road is set to zero; BMMS-derived bridge spans and pre-set zero lengths for source/sink nodes are never overwritten. All lengths are rounded to integer metres. The result is exported as `data/network_data.csv`, which is the direct input to the Mesa/NetworkX simulation model. The choice of experimental factors, replication counts, and output metrics for the subsequent simulation study follows the principles of Sanchez, Sanchez and Wan [2], whose tutorial on simulation experiment design provides guidance on efficiently allocating simulation runs to balance statistical confidence against computational cost — a particularly relevant concern when sweeping bridge-breakdown scenarios across a large road network.

3

Model Design and Implementation

In this chapter, we describe the model's design and implementation based on the modifications to the provided code. The set, namely, contains a script for the initiation of the model, the definition of the infrastructure components, and the running of the simulation.

The model is initiated using the `NetworkX` and `MESA` packages. The relation between how the two packages have been incorporated into the model design is described in the subsequent subsection and a flowchart is provided in Figure 3.1 that shows how road data is read and exchanged between the two in the script.

3.1. Updates in `model.py`

This script contains the initiation of `BangladeshModel`. We use the `NetworkX` package to generate the network graph in the instantiation of the model, where this is done with the line `self.G = nx.Graph()`. Furthermore, a parameter has been added to the model to pass in different breakdown probabilities for each bridge grade.

3.1.1. Reading the roads csv and processing into a network graph

In `generate_model`, where the simulation model is generated. The most crucial change here is the creation of a network graph from the cleaned roads file generated from the steps described in Chapter 3. Firstly, the list of roads are generated as a unique list from the dataframe created from the csv file. Then, a series of IDs is extracted for each road to store the component IDs from its start to the end as a dictionary called `path_ids_dict`. The reverse is also stored to allow vehicles to drive in the opposite direction. Each path is anyways stored as `(start, None)` so that a vehicle can be routed to any point from a given starting point.

The network graph is then constructed using the `NetworkX` package. For each road, the `sourcesink` (which represents the start and the end of a road) is identified as the main body of a road. Edges are then added to the graph between every pair of consecutive components along this main body, with the edge weight set to the length of the component. A minimum of 1 is enforced as shown in `max(float(main_body.iloc[i]['length']), 1)` to prevent zero-weight edges for components with missing or zero lengths. The code also handles the case where a road contains only a single `sourcesink`, in which all consecutive components are connected as edges regardless to ensure completeness.

Finally, to connect the roads together, the csv also includes a cross-reference point representing the intersections. These are connected to either the start or the end of the current road, based on explicit labelling (`LRPS` for start and `LRPE` for end) that determines the connection. Otherwise, connectivity is determined by geographical proximity, based on the distance between the start or end points of the coordinates.

3.1.2. Picking routes and computing shortest path

The function `get_random_routes` picks a random sink destination and computes the shortest path using `NetworkX` with the edge weights corresponding to component lengths. It caches discovered paths in the `path_ids_dict` dictionary for efficient future lookups. However, if no path exists an exception is raised and the method falls back to `get_straight_route` to return a simple direct route along the road the vehicle is currently on.

3.1.3. Recording trip data analytics

While in the previous assignment a DataCollector is used to record trip analytics, a custom function `record_trip` is used this time to collect the relevant information about a trip done by a vehicle. We found that this method is more efficient. The data collected are, among others, the vehicle ID, its origin and destination, the time elapsed and the bridges passed.

The trip data is finally exported as a csv using the `export_data` function.

3.2. Updates in `components.py`

The original component structure was largely preserved, but several modifications were introduced to support condition-based bridge breakdowns, per-vehicle trip tracking, and per-bridge delay analysis.

Bridge. The `Bridge` component was revised to support scenario-driven breakdown behavior. Upon instantiation, each bridge checks its condition category (A–D) against the breakdown probability provided by the model and outlined in table 4.1. If a random draw falls below the corresponding threshold, the bridge is marked as `broken_down`.

Broken bridges then generate a new random delay for each vehicle using the same length-dependent delay distributions defined in Assignment 2. In addition, two accumulators, `total_delay_caused` and `vehicles_delayed`, were introduced. These track each bridge's cumulative impact for the top-5 bridge analysis.

Vehicle. Two new attributes were added to the `Vehicle` component, which are `total_waiting_time` and `bridges_passed`. These are updated in `drive_to_next` whenever a vehicle encounters a bridge.

When a vehicle reaches a sink, `model.record_trip(self)` is called before the vehicle is removed. This stores the vehicle's complete trip record in the model's `trip_data` list, including origin, destination, travel time, cumulative waiting time, and bridges traversed.

Finally, debug print statements in `Vehicle.step`, `Source.generate_truck`, and `Sink.remove` were removed to avoid unnecessary performance degradation during batch experiments.

3.3. Updates in `model_run.py`

The simulation is done in the `model_run.py` script. The function `run_experiments` is created to conveniently execute the simulation experiments. It iterates over each scenario defined in `SCENARIOS`. For every scenario, the simulation is run `NUM_REPLICATIONS` times, each with a fixed seed drawn from a predefined list to ensure reproducibility.

For each replication, `run_single` is called with the corresponding seed, probability configuration, and run length and eventually returns a dataframe of completed trips. Metadata coluns are appended to this dataframe, recording the replication number, seed and scenario name to allow the results to be traced back to the right configuration. Performance metrics recorded by the data collecting function described above such as the number of completed trips, mean travel time and the mean waiting time are printed to the console after each replication.

Finally, once all replications for a scenario are complete, the results are concatenated into a dataframe exported as a csv file.

3.4. Reflection on the model design

This study highlights the advantage of simulation experiments over real-world experiments to decide on which bridges to invest. Sanchez, Sanchez and Wan [2] noted that simulation allows for broader

and more controlled experimentation than would be feasible in practice. It is simply not possible nor desirable to deliberately induce bridge failures in the real world. Moreover, we are also able to simulate over a broader range of severity scenarios.

Replication across multiple seeds is used to account for the inherent stochasticity of the simulation (for example, stemming from the delay time, which is drawn from a distribution). However, it is important to acknowledge that the mean travel and waiting times reported are estimates of the true mean given the model's inputs and assumptions made.

In this study, what defines a critical bridge is the travel time it accumulates at the end of each run. In other words, criticality is the cumulative effect of how many vehicles have to be delayed by a specific bridge breakdown and how long each delay is. We can apply three dimensions that help to evaluate this criticality metric [1]:

- Functionality. A critical bridge will be one that increases overall delay time of vehicles, hindering economic activity.
- Ethical. A critical bridge will likely be one that increases delay time for the most amount of vehicles.
- Aggregation. A critical bridge can only be located in the national roads modeled, so bridges in regional or other national roads are excluded. In addition, since the shortest path is computed based on distance, shortest paths remain the same, and therefore, a critical bridge will only be in a road that is part of the shortest path.

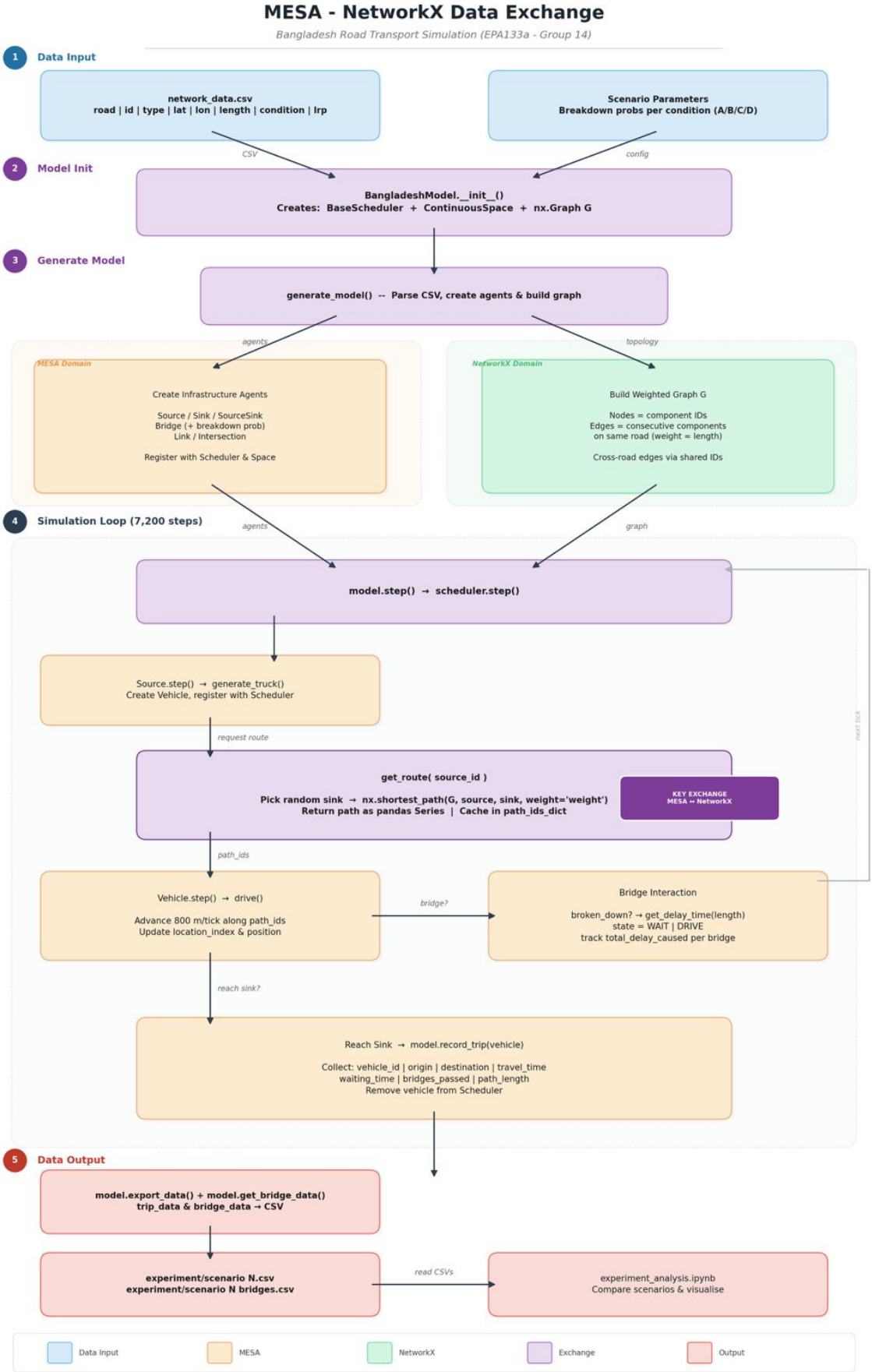

Figure 3.1: Model design showing relation and data exchange between MESA and NetworkX

4

Results and Discussion

4.1. Experimental Setup

The simulation was configured to run five scenarios with increasing bridge breakdown probabilities, as summarised in Table 4.1. Scenario 0 serves as the baseline with no breakdowns; Scenarios 1–4 progressively increase the probability of failure for each condition category.

Table 4.1: Breakdown probability per bridge condition category for each scenario.

Scenario	Cat A (%)	Cat B (%)	Cat C (%)	Cat D (%)
0	0	0	0	0
1	0	0	0	5
2	0	0	5	10
3	0	5	10	20
4	5	10	20	40

Each scenario was run for 10 replications using different fixed seeds to ensure reproducibility while varying which bridges break down. Each replication simulates 5 days of operation (7,200 ticks at 1 tick per minute). Vehicles are generated every 5 minutes from all 18 road SourceSinks and assigned a random destination via the NetworkX shortest-path algorithm. The driving speed is set to 48 km/h, consistent with Assignment 2. No congestion effects are modelled; delays arise exclusively from broken bridges. Visualisation was disabled during batch runs to reduce execution time.

The primary output metrics are *travel time* (total time from generation to arrival at destination, in minutes) and *total waiting time* (cumulative bridge delay per vehicle, in minutes). All results are exported as CSV files and analysed in `experiment_analysis.ipynb`.

4.2. Results

Figure 4.1 presents the distribution of travel time across all five scenarios. Under baseline conditions (Scenario 0), the mean travel time is 3.37 hours. Travel time remains virtually unchanged in Scenario 1 (mean 3.38 hours), indicating that a 5% breakdown probability applied only to Cat D bridges has negligible system-wide impact. From Scenario 2 onward, a clear escalation emerges: the mean increases to 3.53 hours, then 4.40 hours in Scenario 3, and 6.17 hours in Scenario 4, nearly doubling the baseline. The interquartile range also widens substantially in Scenarios 3 and 4, with upper whiskers reaching 12 and 16 hours respectively, indicating that some vehicles experience severe delays. This widening reflects a key property of network routing: depending on which random destination a vehicle is assigned, it may traverse heavily disrupted corridors or avoid them entirely. As a result, the spread of outcomes is much broader than in the baseline scenario, where all delays are zero.

Figure 4.2 shows the percentage of trips affected by bridge delays. While only 2.4% of trips experience

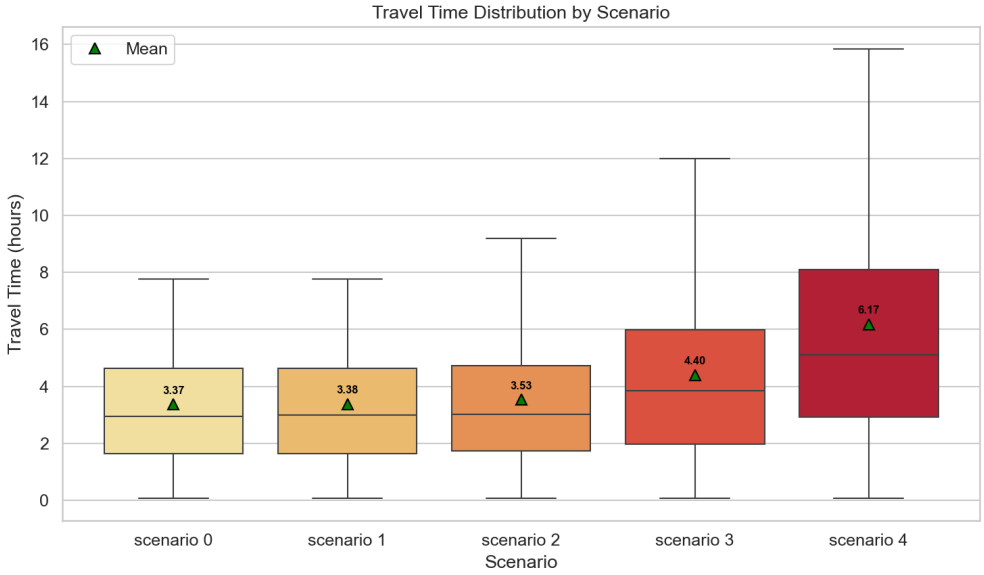

Figure 4.1: Travel time distribution by scenario. Green triangles indicate the mean. Scenario 4 nearly doubles the baseline mean travel time, with a substantially wider spread.

any delay in Scenario 1, this rises to 27.8% in Scenario 2, 79.1% in Scenario 3, and 92.6% in Scenario 4. This non-linear escalation suggests that once a critical mass of bridges is disrupted, the network effect amplifies the impact. Vehicles travelling across multiple roads may encounter broken bridges along different segments of their route. This observation aligns with Jafino, Kwakkel, and Verbraeck [1], who note that criticality in transport networks depends not only on individual component properties, but also on network position and traffic exposure.

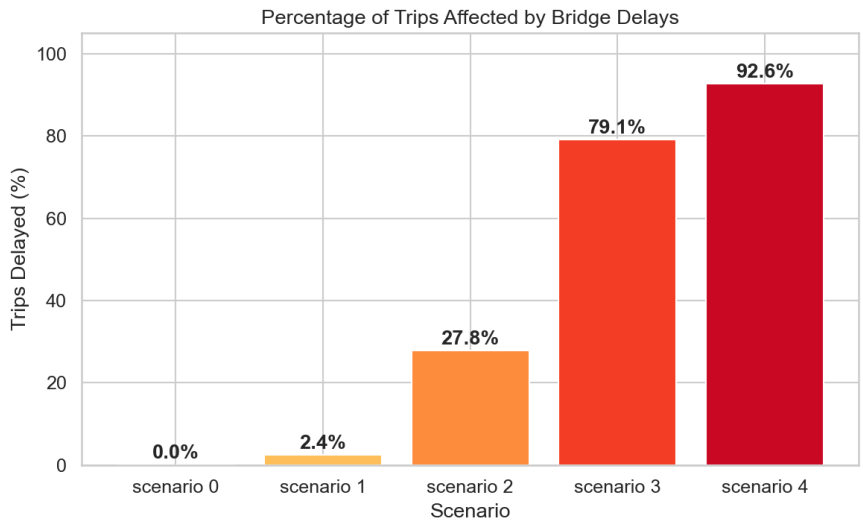

Figure 4.2: Percentage of trips affected by bridge delays per scenario. The non-linear escalation from 2.4% (Scenario 1) to 92.6% (Scenario 4) illustrates the network-level propagation of disruptions.

The top five bridges identified for investment, ranked by average total delay across Scenarios 1–4 with equal weighting, are shown in Figure 4.3. Shalipur Bridge on N2 dominates with 887 hours of average cumulative delay, followed by the Bride over Arial RCC Gider Bridge (592 h), Old Brammapulra (570 h), Purpu Durgapure Box Culvert on N1 (516 h), and Sonarampur Bridge (441 h). Notably, four of the five bridges are located on N2, and all five share condition category C. This suggests that Cat C bridges

on high-traffic corridors are the primary bottleneck.

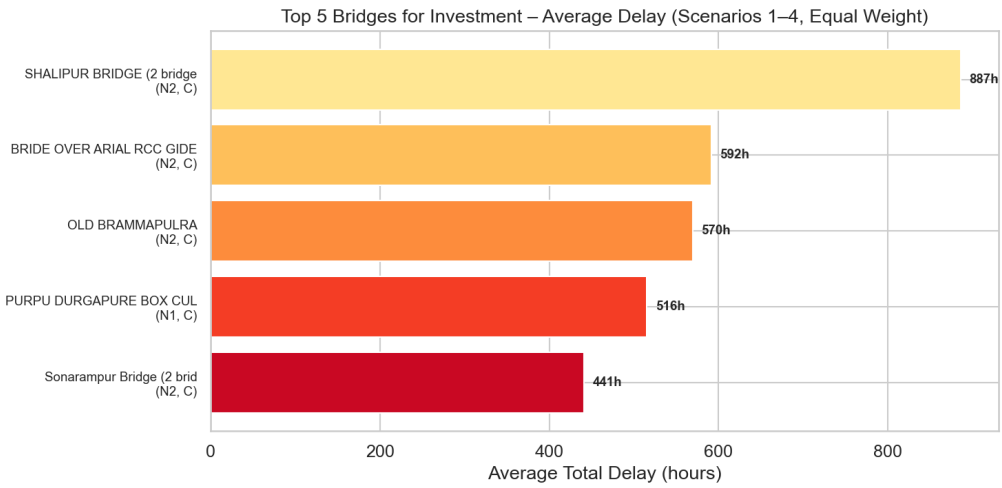

Figure 4.3: Top 5 bridges for investment ranked by average total delay across Scenarios 1–4 (equal weight). Four of the five are located on N2.

Figure 4.4 shows the geographic location of these bridges on the network. Four of the five are concentrated along the central and northern segments of N2, where multiple side roads (N102, N204, N207, N208) branch off. This clustering is significant because vehicles routed between N1 endpoints and the eastern side roads must traverse N2 through these segments, which amplifies the impact of any disruption. This is consistent with the finding from [1] that simple structural attributes alone do not determine criticality. Network position and traffic volume also play important roles.

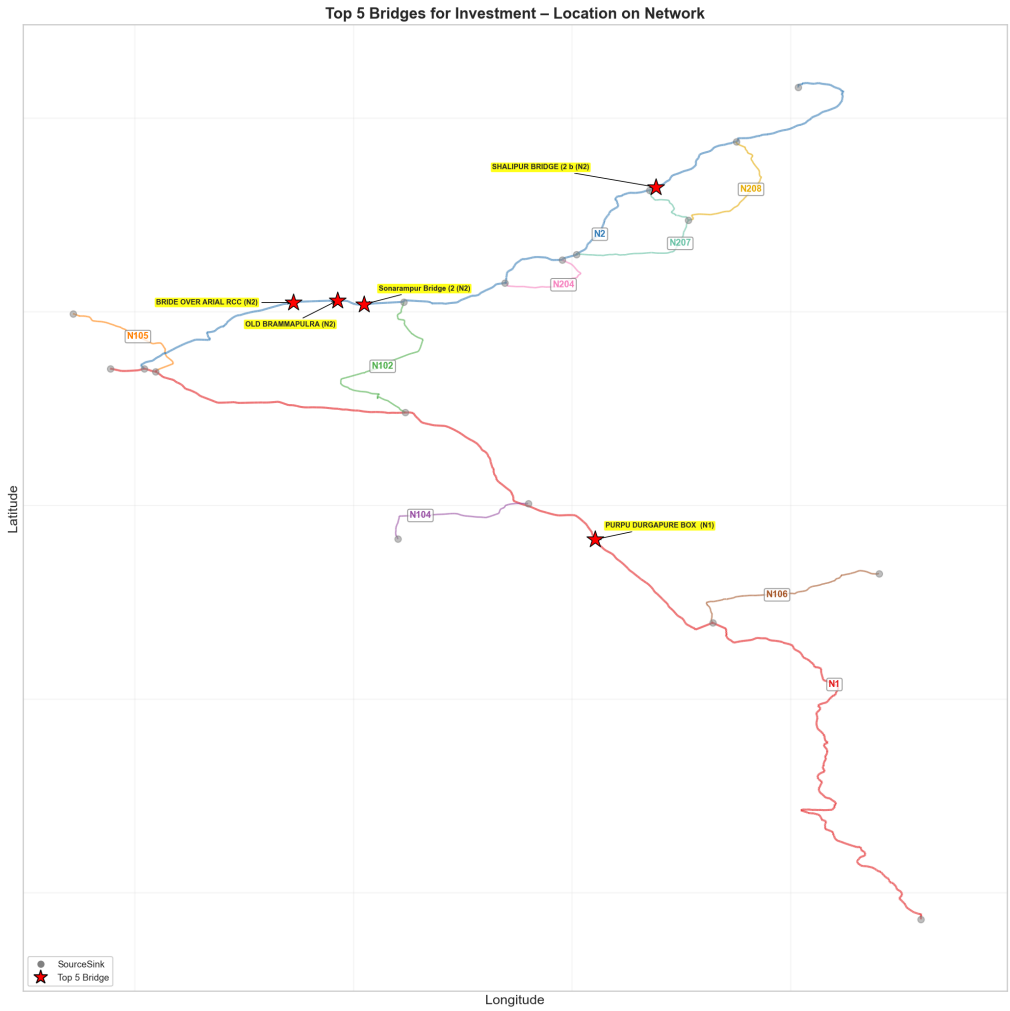

Figure 4.4: Geographic location of the top 5 bridges (red stars) on the road network. The clustering on N2 near side-road intersections explains their disproportionate impact on system-wide delay.

We can highlight that one bridge is part of the top five candidates identified in this report and on the Assignment 2 report: PURPU DURGAPURE BOX CUL (N1, C). This double appearance signals the culvert as good candidate for investment.

Sanchez, Sanchez, and Wan [2] highlight the importance of replication in simulation studies to account for stochastic variability. Our use of 10 replications per scenario, each with different bridge breakdown configurations, reveals meaningful variance, particularly in Scenarios 3 and 4, where the specific bridges that break down substantially affect the outcome.

4.3. Bonus Questions

4.3.1. Bonus 1: Accurate Intersection Locations Using Road Geometries

Motivation

In the main data-preparation pipeline, intersection nodes were placed at the coordinates recorded in `_roads3.csv` for `CrossRoad` and `SideRoad` entries. These are field-survey observations: a surveyor noted a crossing point while travelling along a road, so the coordinates reflect the surveyor's position rather than the geometric intersection of the two road centrelines. A more principled approach is to compute intersections directly from the road geometries.

Data Source

The folder `data/gis/osm/` contains an OpenStreetMap (OSM) road-network shapefile (`roads.shp`) in the WGS 84 coordinate reference system (EPSG:4326). Each feature is a road segment annotated with a `ref` attribute (e.g. `"N 1"`, `"NH208"`, `"N 105;N 106"`) that identifies the national highway to which the segment belongs.

Method

The following procedure was implemented in the data-preparation notebook using `GeoPandas` and `Shapely`:

1. **Load and normalise.** The shapefile was loaded with `geopandas.read_file`. OSM `ref` strings were normalised to a canonical form (e.g. `"N 1"` → `"N1"`, `"NH208"` → `"N208"`) using a regular expression, accounting for semicolon-separated multi-road entries.
2. **Merge segments per road.** All segments belonging to the same road were combined into a single geometry with `shapely.ops.unary_union`, producing one `LineString` or `MultiLineString` per road.
3. **Compute pairwise intersections.** For every pair of roads that share an intersection node in the model, Shapely's `.intersection()` operator was applied to the two merged road geometries. A true geometric intersection returns a `Point` object whose coordinates are used directly.
4. **Fallback for coverage gaps.** Where two roads do not share a common vertex in the OSM data (i.e. `.intersection()` returns an empty geometry), `shapely.ops.nearest_points` was used to find the closest pair of points between the two geometries. The midpoint of that pair was taken as the junction location, provided the gap was less than 200 m; otherwise the pair was flagged as unresolvable via the OSM method.

Visualisation

Figure 4.5 shows the result of the comparison. Each of the nine modelled roads is drawn in a distinct colour using the survey-point centreline reconstructed from `_roads3.csv`. Intersection nodes are plotted in one of three categories: a red $\times$ paired with a green $\circ$ for junctions found in both the model and OSM (the distance between them is annotated); an amber $\times$ for model-only junctions whose road pair could not be located in the OSM shapefile; and an amber $\circ$ for any OSM-only junction with no corresponding model node (none were observed in this dataset).

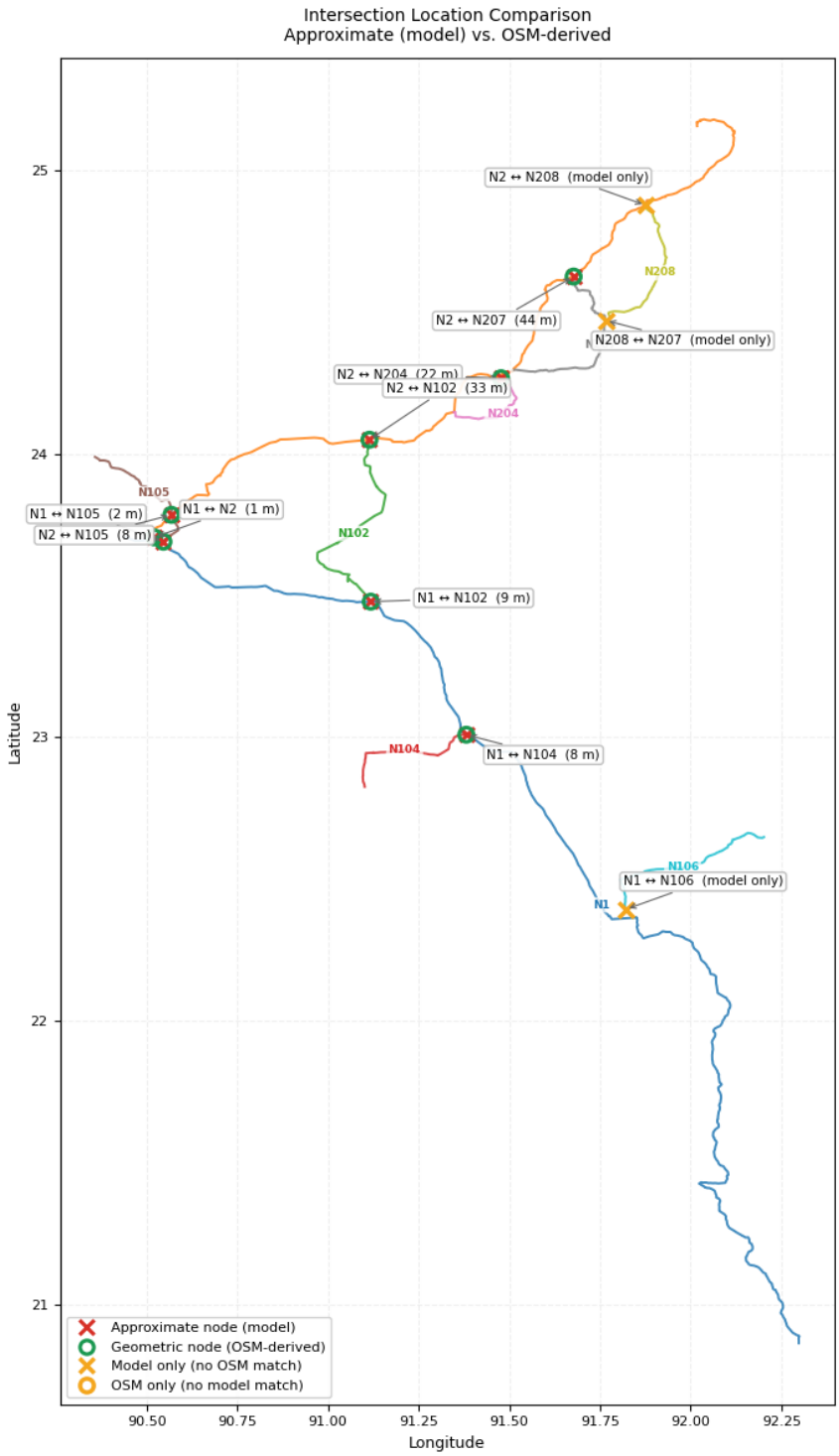

Figure 4.5: Intersection location comparison: approximate model nodes (red ×) versus OSM-derived geometric nodes (green ○). Amber × markers indicate junctions present in the model but undetectable from the OSM shapefile. Roads are coloured by name; dotted lines connect matched pairs that are visibly offset.

4.3.2. Bonus 2: Impact of Intersection Location Accuracy on Experimental Outcomes

Comparison Results

Table 4.2 summarises the offset between each OSM-derived intersection point and the corresponding approximate location used in the model. Three qualitatively different cases were identified.

Table 4.2: Positional offsets between OSM-derived and approximate intersection locations.

Road pair	Offset (m)	Remark
N1 ↔ N2 (primary junction)	1	Clean geometric crossing
N1 ↔ N105	2	Clean geometric crossing
N1 ↔ N104	8	Clean geometric crossing
N2 ↔ N105	8	Clean geometric crossing
N1 ↔ N102	9	Clean geometric crossing
N2 ↔ N204	22	Partial OSM coverage
N2 ↔ N102	33	Partial OSM coverage
N2 ↔ N207	44	Partial OSM coverage
N1 ↔ N106	—	Model only (no OSM geometry)
N207 ↔ N208	—	Model only (no OSM geometry)
N2 ↔ N208	—	Model only (no OSM geometry)

Case 1 - Clean intersections ($\leq$9 m offset). For five road pairs — N1 ↔ N2, N1 ↔ N105, N1 ↔ N104, N2 ↔ N105, and N1 ↔ N102 — the OSM road centrelines genuinely cross in the shapefile and the positional offset is at most 9 m. At the modelled speed of 48 km/h, a 9 m offset corresponds to less than 0.001 minutes of extra travel time per vehicle crossing that node.

Case 2 - Partial OSM coverage (22–44 m offset). For N2 ↔ N204 (22 m), N2 ↔ N102 (33 m), and N2 ↔ N207 (44 m), the OSM shapefile maps only partial segments of the side roads; the nearest-points fallback finds the closest OSM endpoint, which lies within 44 m of the survey-crossing location. Despite the partial coverage, all three offsets are well within the acceptable range for this macroscopic model. At 48 km/h, a 44 m offset introduces less than 0.055 seconds of additional travel time — negligible relative to bridge-delay effects.

Case 3 - N1 ↔ N2 parallel section. N1 and N2 run alongside each other for approximately 10 km south of Dhaka before diverging. The OSM shapefile records five shared vertices or crossings along this stretch. Our model places a single intersection node at the primary (northernmost) junction, which matches the first OSM crossing to within 1 m. The additional crossings reflect the parallel-running section and require no separate network nodes.

Case 4 - Junctions undetectable via the OSM method. Three junctions could not be resolved from the OSM shapefile and appear as model-only nodes (amber $\times$) in Figure 4.5. The OSM coverage of N208 spans approximately 23.84°N–24.18°N, while N207 is mapped between 24.49°N–24.63°N; the two geometries are geographically disjoint. The N106 shapefile coverage ends well before the road's starting point at Oxygen More (22.39°N, 91.82°E), which lies within the same Chittagong city gap that also interrupts the N1 survey data. The N1 ↔ N106, N207 ↔ N208, and N2 ↔ N208 junctions are real and were correctly placed using the survey-crossing method; the OSM data simply lacks sufficient coverage to locate them geometrically.

Impact on Simulation Outcomes

The simulation uses intersection coordinates for two distinct purposes: (1) spatial placement on the `ContinuousSpace` canvas (visualisation only), and (2) network topology - which road segments are connected for shortest-path routing via NetworkX.

Topology is unaffected. Graph connectivity is determined by which roads are declared to share an intersection node, not by the node's exact coordinates. Routing decisions and travel paths are therefore

identical regardless of whether an intersection node is placed at the precise geometric crossing or 44 m away.

Segment lengths change negligibly. The largest observed offset is 44 m, redistributing at most 44 m of length between two adjacent road segments that are typically 500–2 000 m long, introducing a relative error below 9 %. At 48 km/h this translates to less than 0.055 seconds of additional travel time per segment per vehicle.

Bridge delays dominate. The experimental scenarios vary bridge breakdown rates from 0 % (baseline) to 40 % for Category D bridges. A single disabled bridge introduces a random delay of several simulation ticks (minutes); this is orders of magnitude larger than the sub-second effect of a 44 m intersection offset. Intersection placement accuracy is therefore irrelevant to the experimental outcomes.

OSM completeness matters more than positional accuracy. The comparison reveals that the OSM shapefile lacks complete coverage for N204, N207, and N102 along their southern extents, and has no usable geometry for N106, N207–N208, and N2–N208 junctions. For these cases, the survey-crossing approximation is the more reliable source. This highlights that uncritical reliance on a single data source - even official-looking GIS data - can introduce errors that field-survey data avoids.

Conclusion

All eight matched intersection pairs fall within 44 m of their OSM-derived counterparts, with five pairs under 10 m. These offsets produce travel-time differences well below 0.1 seconds per segment — negligible relative to the minutes-scale delays caused by bridge breakdowns. For the three model-only junctions (N1–N106, N207–N208, N2–N208), OSM coverage is simply absent; the survey-point locations are the best available source. A precision improvement from purely GIS-derived locations would only be material for micro-simulation tasks sensitive to sub-50 m positional accuracy, such as turning-movement modelling at junctions, which is beyond the scope of this macroscopic network model.

5

Conclusion and Reflection

5.1. Conclusion

This study set out to automatically generate a Mesa model to simulate vehicle movements and delay time along a major road network in Bangladesh (N1, N2 and national side roads), where delay time was formulated as a function of length and bridge breakdown status. The main objective was to identify the bridges in the network that generated the highest waiting time across multiple scenarios to select top candidate bridges for investment. Based on the results obtained, the modeling and simulation goals were achieved.

The component-based modeling approach proved effective, as most components from Assignment 2 could be reused with only minor adjustment to the vehicle component to support improved data collection. Modifications were concentrated in the `generate_model` and `get_random_route` functions to build the network and compute the shortest path with the `NetworkX` package. Additional effort was dedicated to preparing the data so the model could accurately generate the infrastructure components of the network. Furthermore, the experimental setup was strengthened through the use of fixed seeds and multiple simulation runs, improving the reliability and replicability of the results. Finally, figures of key components of the network and of data results graphs were added to contribute to a more comprehensive analysis.

The top five bridges identified represent priority candidates for targeted investment for improvements. And attention could be placed on box culvert PRUPU DURGAPURE in N1, which appeared as a top candidate in this report and our previous Assignment 2 report.

5.2. Limitations

Since the paths taken by the trucks are selected by either straight routes or shortest paths based on distance, the paths will not change when different bridges breakdown. However, in reality there could be alternate routes that a vehicle might take when a bridge creates delay time. This is particularly relevant because it reflects the importance of connectivity, and could change the results of the Top bridge candidates for investment. For example, according to Figure 4.3, SHALIPUR BRIDGE (2 bridge N2, C) is the most important candidate since it contributes to a significant delay, however, in Figure 4.4 we observe that a path set to travel through this bridge might take alternate routes. The current design does not allow a behavior to take alternate routes, which is a model limitation.

On broader terms, we only considered a handful of parameters to influence criticality of the network: aggregation of only national roads, delay time, bridges break down with probability based on condition. Yet, there are hundreds more parameters that also influence vulnerability and criticality of bridges in Bangladesh.

5.3. Future Improvements

We could continue adding more parameters that also influence the vulnerability and/or criticality of the road network such as vulnerability to flooding, add regional and national roads, add volume of goods transported, add a transportation pattern, among others. However, as we add parameters and complexity to the model, the model run will call for more computing power. Experiments on models with these many input factors benefit from sensitivity analysis to identify which parameters are worth modeling, and design of experiments to generate more efficient experimentation outputs and analysis [2].

References

[1] Bramka Arga Jafino, Jan Kwakkel, and Alexander Verbraeck. “Transport network criticality metrics: a comparative analysis and a guideline for selection”. In: *Transport Reviews* 40.2 (2020), pp. 241–264.

[2] Susan Sanchez, Paul Sanchez, and Hong Wan. “Work smarter, not harder, a tutorial on desigining and conducting simulation experiments”. In: *Proceedings of the 2021 Winter Simulation Conference* (2021).

A

Data and Code Availability

All data processing, analysis, and visualization code used in this study are openly available to ensure transparency and reproducibility. The complete codebase, along with documentation and links to the underlying datasets, is hosted in a public GitHub repository at:

`https://github.com/taufik-my/ASIM_Team14/tree/main/EPA133a-G14-A3`

B

Use of AI Tools

B.1. How AI Was Used

- AI (Gemini and ChatGPT) was used to improve writing style and spell check. Grammarly was also used to review our writing. We reviewed the output to ensure consistency with our writing style.
- AI (Gemini and ChatGPT) was used to reformat our writing in LaTeX where needed.
- AI (Gemini, ChatGPT, and Claude) was used to help debug issues in our code.

B.2. How AI Was Not Used

- AI was not used to generate the final results or interpret them.
- AI was not used to identify errors in the dataset.
- AI was not used to make unverified claims.

C

Team Contributions

This assignment was completed collaboratively by all members of Team 14. The table below summarises the primary contributions of each team member. All members participated in discussions, validation of results, and revision of the final report.

Table C.1: Overview of team member contributions

Student Name	Main Contribution
Bayu Jamalullael	Data Preparation; cleaning and combining road and bridge dataset to be a model-ready dataframe; bonus exercises
Brenda Escobar Arriaga	Introduction, collaboration for networkx implementation, model design reflection, and conclusion
Brian Parsaoran	Data preparation, model integration
Taufik M. Yusup	Model integration, results, data visualization
Zhafran Sidik	Component and model architecture, and writing